CISS245: Advanced Programming
Quiz q2903

Name: eagraham1@cougars.ccis.edu Score:

Q1. Complete the following to store value `v` at the integers from address `start` up to but not including `end`. Recurse on `start`, i.e., in your recursive case for `fill(start, end, v)`, you should call `fill(start + 1, end, v)`. (5 seconds. Test.)
Answer:

```
void fill(int * start, * end, int v)
{
  if(start == end) return;

  *start = v;

  fill(start + 1, end, v);
}
```

Q2. This is the same as above, except that you recurse on `end`, i.e., in your recursive case for `fill(start, end, v)`, you should call `fill(start, end - 1, v)`. (5 seconds. Test.)
Answer:

```
void fill(int * start, * end, int v)
{
  if (start == end) return;

  fill(start, end - 1, v);

  *(end - 1) = v;
}
```

Q3. This is the same as above, except that you recurse using divide-and-conquer. (20 seconds. Test.)
Answer:

```
void fill(int * start, * end, int v)
{
  if (start == end) return;

  int* mid = start + (end - start) / 2;

  fill(start, mid, v);
  fill(mid, end, v);
```

```
}
```

Q4. Complete the following function that counts the number of times `target` occurs in the integers from address `start` up to but not including `end`. Recurse at `start`. (5 seconds.)
Answer:

```
int count(int * start, int * end, int target)
{
  if(start == end) return 0;
  return (*start == target) + count(start + 1, end, target);
}
```

Q5. Complete the following function that counts the number of times `target` occurs in the integers from address `start` up to but not including `end`. Recurse at `end`. (5 seconds.)
Answer:

```
int count(int * start, int * end, int target)
{
  if(start == end) return 0;
  end--;

  return (*end == target) + count(start, end, target);
}
```

Q6. Complete the following function that counts the number of times `target` occurs in the integers from address `start` up to but not including `end`. You must use divide-and-conquer. (20 seconds.)
Answer:

```
int count(int * start, int * end, int target)
{
  if (start >= end) return 0;

  int * mid = start + (end - start) / 2;
  return count(start, mid, target) + count(mid, end, target);
}
```

Q7. Complete the `bubblesort` function and the `bubblesort_pass` function. This was already covered in class. So this is a freebie. Finish the following bubblesort function by completing all the functions using recursion. Do not look at your class notes and try to do is on your own. If you are lost, *then* look at your notes. There must be no loops in the program. Your `bubblesort` must call the `bubblesort_pass` function that executes one pass of the `bubblesort`.

Answer:

```
#include <iostream>

void swap(int * x, int * y)
{
  int * temp = *x;
  *x = *y;
  *y = *temp;
}

void bubblesort_pass(int * start, int * end)
{
  if (start >= end - 1) return;

  if (*start > *(start + 1)) swap(start, start + 1);

  bubblesort_pass(start + 1, end);
}

void bubblesort(int * start, int * end)
{
  if (start >= end - 1) return;

  bubblesort_pass(start, end);
  bubblesort(start, end - 1);
}

void println(int * start, int * end)
{
    if (start >= end - 1)
    {
        std::cout << '\n';
    }
    else
    {
        std::cout << *start << ' ';
        println(start + 1, end);
    }
}

int main()
{
    int x[] = {9, 7, 5, 3, 1, 0, 2, 4, 6, 8};
    bubblesort(&x[0], &x[10]);
    println(&x[0], &x[10]);
    return 0;
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.